

KOVA

RV32I · SKY130 · 130nm

Open-source 32-bit RISC-V microcontroller — RTL to silicon tapeout

Built from scratch by a solo ECE student · Kolkata, India

PHASE 1 · 6–8 weeks

Verilog Mastery

Write synthesizable RTL confidently.

Learn the language of hardware.

START HERE

LEARN

- HDLBits — do every single problem → hdlbits.01xz.net
- Verilog by NPTEL (IIT) — free, structured lecture series
- Verilog HDL — Samir Palnitkar (free PDF)

INSTALL TOOLS

- Icarus Verilog — simulator (free) → iverilog.icarus.com
- GTKWave — waveform viewer (free) → gtkwave.sourceforge.net
- EDAPlayground — browser-based, no install needed

BUILD

- ◆ ALU (add, sub, AND, OR, shift)
- ◆ 32-register file (32-bit wide)
- ◆ FSM-based traffic light + UART
- ◆ Testbenches for every module

AFTER THIS PHASE YOU WILL:

Write any combinational/sequential circuit. Simulate and debug waveforms. Think in hardware.

PHASE 2 · 6–8 weeks

Computer Architecture

Understand exactly what you are

building before you build it.

READ

- Computer Organization & Design RISC-V Edition — Patterson & Hennessy (MANDATORY)
- RISC-V Specification — download RV32I base spec, read fully → riscv.org/technical/specifications

WATCH

- Computer Architecture — ETH Zurich → YouTube playlist

UNDERSTAND

- ◆ Fetch → decode → execute → memory → writeback pipeline
- ◆ All 47 RV32I instructions
- ◆ Data hazards and control hazards
- ◆ Memory-mapped I/O concept

AFTER THIS PHASE YOU WILL:

Draw a CPU datapath from memory. Explain every RV32I instruction. Read the spec and understand every line.

PHASE 3 · 8–10 weeks

Build the RISC-V Core

Working RV32I CPU in Verilog — fully simulated and tested.

■ KOVA PROJECT STARTS HERE

WATCH

- Building a RISC-V CPU — Robert Baruch (YouTube) — step by step
- RISC-V CPU in Verilog — Digital Design HMC
- PicoRV32 source code — read as reference, do not copy → github.com/YosysHQ/picorv32

BUILD IN ORDER

1. Instruction fetch unit
2. Decoder (all RV32I instructions)
3. Register file
4. Execute stage (ALU)
5. Memory access stage
6. Writeback stage
7. Connect into single-cycle CPU

VERIFY

- ◆ Run RISC-V compliance test suite → github.com/riscv-non-isa/riscv-arch-test
- ◆ Write testbenches for every module
- Paper: "RISC-V: The Free and Open RISC ISA" — Waterman et al. 2014

AFTER THIS PHASE YOU WILL:

Working RV32I CPU in simulation. All 47 instructions passing compliance tests. First real KOVA commit.

PHASE 4 · 4–6 weeks

FPGA Validation

Prove KOVA works in real hardware before spending money on silicon.

FIRST REAL HARDWARE

BUY HARDWARE

- Arty A7-35T (~\$11,000) — recommended
- iCESugar (~\$2,500) — budget option

INSTALL TOOLS

- Xilinx Vivado WebPACK (free) → [xilinx.com](https://www.xilinx.com)
- Yosys + nextpnr (open source) → github.com/YosysHQ/yosys

MILESTONES

- ◆ Boot LED blink via memory-mapped GPIO
- ◆ Run basic arithmetic programs
- ◆ Print "Hello from KOVA" over UART ← this is the moment

AFTER THIS PHASE YOU WILL:

KOVA running on real hardware. Confidence that your design is correct. CPU feels alive.

PHASE 5 · 6–8 weeks

MCU Peripherals

Wrap the CPU with peripherals to make it a true microcontroller.

STUDY

- PicoRV32 + SoC example → github.com/YosysHQ/picorv32
- LiteX SoC builder → github.com/enjoy-digital/litex
- Paper: "Rocket Chip Generator" — Asanovi et al., UCB

BUILD PERIPHERALS

- ◆ GPIO controller
- ◆ UART module
- ◆ SPI controller
- ◆ Timer / PWM
- ◆ On-chip SRAM
- ◆ Interrupt controller

AFTER THIS PHASE YOU WILL:

KOVA is a complete microcontroller — comparable in structure to ATmega or STM32, but built by you.

PHASE 6 · 8–10 weeks

Physical Design (VLSI Flow)

Take RTL → GDSII. The file you send to the fab.

RTL → SILICON STARTS HERE

FULL FLOW

RTL (Verilog) → Synthesis → Floorplan → Place & Route → STA → DRC/LVS → GDSII

INSTALL TOOLS

- OpenLane (full flow) → github.com/The-OpenROAD-Project/OpenLane
- SkyWater 130nm PDK → github.com/google/skywater-pdk
- Magic VLSI layout editor → opencircuitdesign.com/magic
- KLayout GDSII viewer → klayout.de
- Yosys (synthesis), OpenROAD (P&R;), Netgen (LVS)

LEARN

- VSD OpenLane Workshop series (YouTube) — best resource for this phase
- OpenLane Documentation → openlane.readthedocs.io
- "Towards Open-Source PD" — Shalan & Edwards 2020
- "OpenROAD" — Ajayi et al. 2019

RUN THE FLOW

1. Synthesize with Yosys
2. Floorplan
3. Place & route with OpenROAD
4. Static timing analysis — achieve timing closure
5. DRC clean (zero violations)
6. LVS clean
7. Generate GDSII ✓

AFTER THIS PHASE YOU WILL:

DRC-clean, LVS-clean GDSII of KOVA. Full understanding of the chip design flow.

PHASE 7 · 4–6 weeks

Tapeout via Efabless

Submit KOVA for real silicon fabrication on SkyWater 130nm.

TO THE FAB

PLATFORM

- Efabless chipignite → efabless.com/chipignite
- ◆ SkyWater 130nm or GF180nm process
- ◆ Sponsored MPW runs = free or very low cost
- ◆ You get actual packaged chips back

STUDY FIRST

- Caravel harness wrapper → github.com/efabless/caravel
- Zero to ASIC course projects → zerotoasiccourse.com
- Matt Venn ASIC designs → github.com/mattvenn

SUBMISSION CHECKLIST

- Design passes DRC
- Design passes LVS
- Timing closure achieved (STA clean)
- Caravel harness integration complete
- Functional simulation passes
- Submit to MPW shuttle

AFTER THIS PHASE YOU WILL:

KOVA is in fabrication. You have done something most engineers never do.

PHASE 8 · After fab return

Chip Back — Test & Document

Hold KOVA in your hand. Power it on. Tell the world.

FINISH LINE

HARDWARE

- ◆ Design PCB to interface your chip (KiCad — free)
- ◆ Power it on. Watch KOVA boot.
- ◆ Test all peripherals

DOCUMENT

- ◆ Push all RTL + flow scripts to GitHub
- ◆ Write a technical paper / blog post
- ◆ Share on RISC-V forums, Efabless community, LinkedIn

AFTER THIS PHASE YOU WILL:

A real chip. Your name on it. Career-defining achievement.

FULL TIMELINE

Phase	Topic	Duration
P1	Verilog Mastery	6–8 weeks
P2	Computer Architecture	6–8 weeks
P3	RISC-V Core (Simulation)	8–10 weeks
P4	FPGA Validation	4–6 weeks
P5	MCU Peripherals	6–8 weeks
P6	Physical Design + OpenLane	8–10 weeks
P7	Tapeout Prep + Submission	4–6 weeks

P8	Chip Test & Documentation	After return
Total		~18–24 months

KEY PAPERS (read in order)

1. The RISC-V Instruction Set Manual — Waterman et al. — riscv.org/technical/specifications
2. Computer Organization & Design RISC-V Edition — Patterson & Hennessy — mandatory textbook
3. Towards Open-Source Physical Design — Shalan & Edwards, 2020
4. OpenROAD: Toward a Self-Driving Open-Source Layout Tool — Ajayi et al., 2019
5. A 32-bit RISC-V Microcontroller (student tapeout) — Search Google Scholar for recent examples

COMMUNITIES TO JOIN NOW

RISC-V International Forum groups.google.com/a/groups.riscv.org

Efabless Discord discord.gg/efabless

Zero to ASIC Community discord.gg/zerotoasic

Reddit [r/FPGA](https://www.reddit.com/r/FPGA) · [r/chipdesign](https://www.reddit.com/r/chipdesign)

A real chip. Your name on it.

18–24 months of disciplined work. One of the rarest achievements a solo ECE student can claim anywhere in the world.

"Most people plan. Few people build. You are building KOVA."